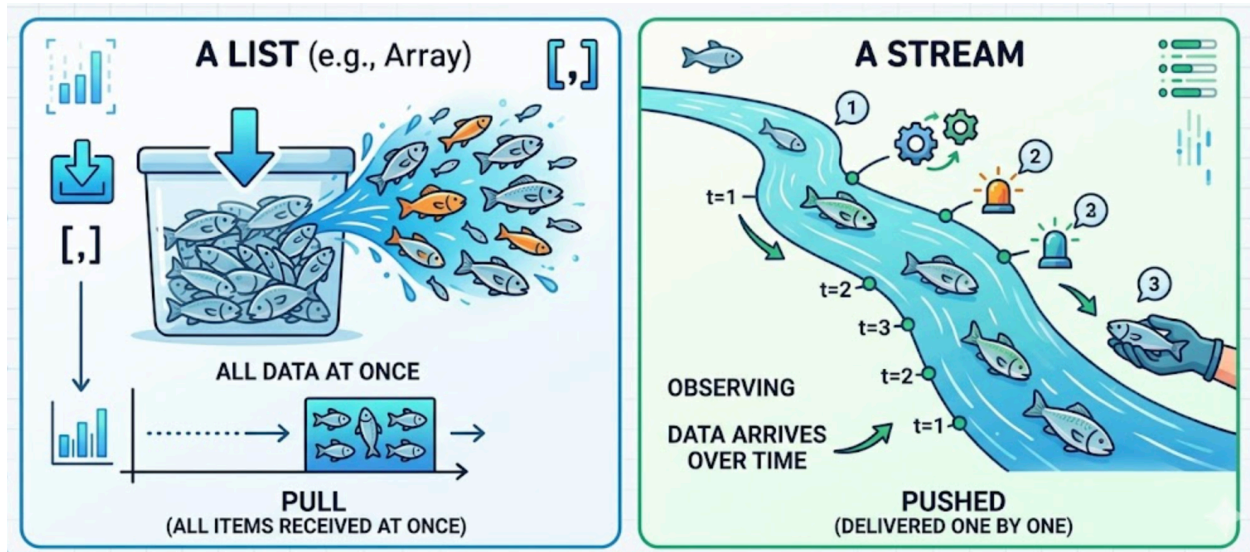


Project Reactor + Flux & Mono

What Are Streams?



List: You get all items at once.

Stream: Items keep coming one by one

A **stream** is a sequence of **events/data** delivered over time. Think of it as a pipe or conveyor belt where values flow continuously, rather than a static list you pull from all at once. e.g.:

- ChatGPT/Claude chats arriving in chunks
- Stock price updates every second
- A file being downloaded chunk by chunk
- Rows coming off a database cursor

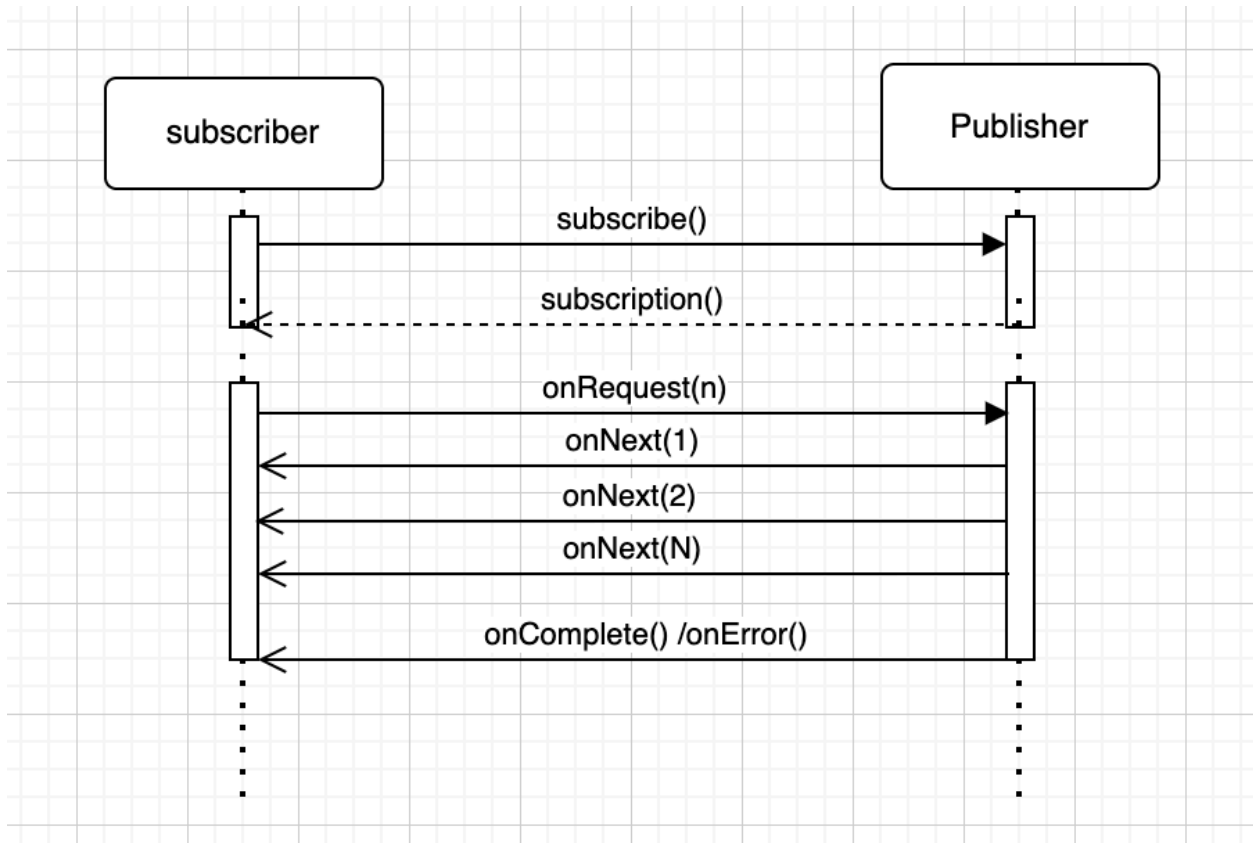
Reactive Streams Specification with Project Reactor

Project Reactor is the reactive library that Spring is built on. Here is a list of what all you can do with it:

- Implement the Reactive Streams Specification

- It gives you two concrete publisher types: `Mono` and `Flux`
- It gives you operators to transform streams
- It is the library underneath everything in Spring WebFlux

And it is best For: Web applications, microservices, Spring Boot projects with Reactive Programming.



The specification dictates four fundamental building blocks that govern how data moves between a producer and a consumer.

Publisher

```

public interface Publisher<T> {
    void subscribe(Subscriber<? super T> subscriber);
}

```

In Reactor, `Flux` and `Mono` both implement `Publisher`.

- **Publisher Type:** `Mono<T>`

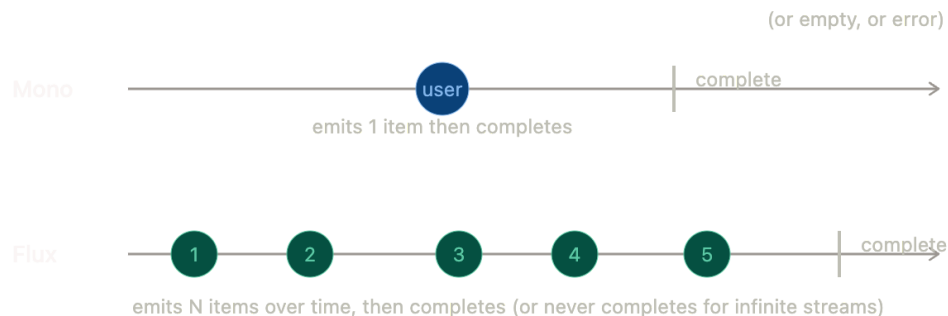
`Mono` represents an asynchronous computation that will eventually produce at most one value, or an error, or just a completion signal (no value).

Think of it as an async `Optional`. Use it wherever you'd return a single object in Spring MVC.

```
// Creating a Mono
Mono<String> hello    = Mono.just("Hello World");
Mono<String> empty    = Mono.empty();           // completes with no value
Mono<String> err      = Mono.error(new RuntimeException("Oops"));

// Real-world usage in a repository
Mono<User> user = userRepository.findById(42L);

// When a Mono emits nothing (empty), switchIfEmpty is your friend:
userRepository.findById(id)
    .switchIfEmpty(Mono.error(new UserNotFoundException(id)));
```



- **Publisher Type:** `Flux<T>`

`Flux` is a stream. It can emit many items over time and eventually complete (or error). Think of it as an async `List` — but one that may never end.

```
// Creating a Flux
Flux<String> items    = Flux.just("A", "B", "C");
Flux<Integer> range   = Flux.range(1, 100);
Flux<Long> interval   = Flux.interval(Duration.ofSeconds
(1)); // ticks forever

// From a list
List<User> users = List.of(user1, user2, user3);
Flux<User> userFlux = Flux.fromIterable(users);
```

Subscriber

```
public interface Subscriber<T> {
    void onSubscribe(Subscription subscription);
    void onNext(T item);
    void onError(Throwable error);
    void onComplete();
}
```

A `Mono` or `Flux` is a blueprint. It describes what to do, but does nothing until someone subscribes.

The `subscribe()` method is how you **start** consuming a reactive stream. It connects the Publisher (Flux/Mono) to a Subscriber and triggers the flow of data.

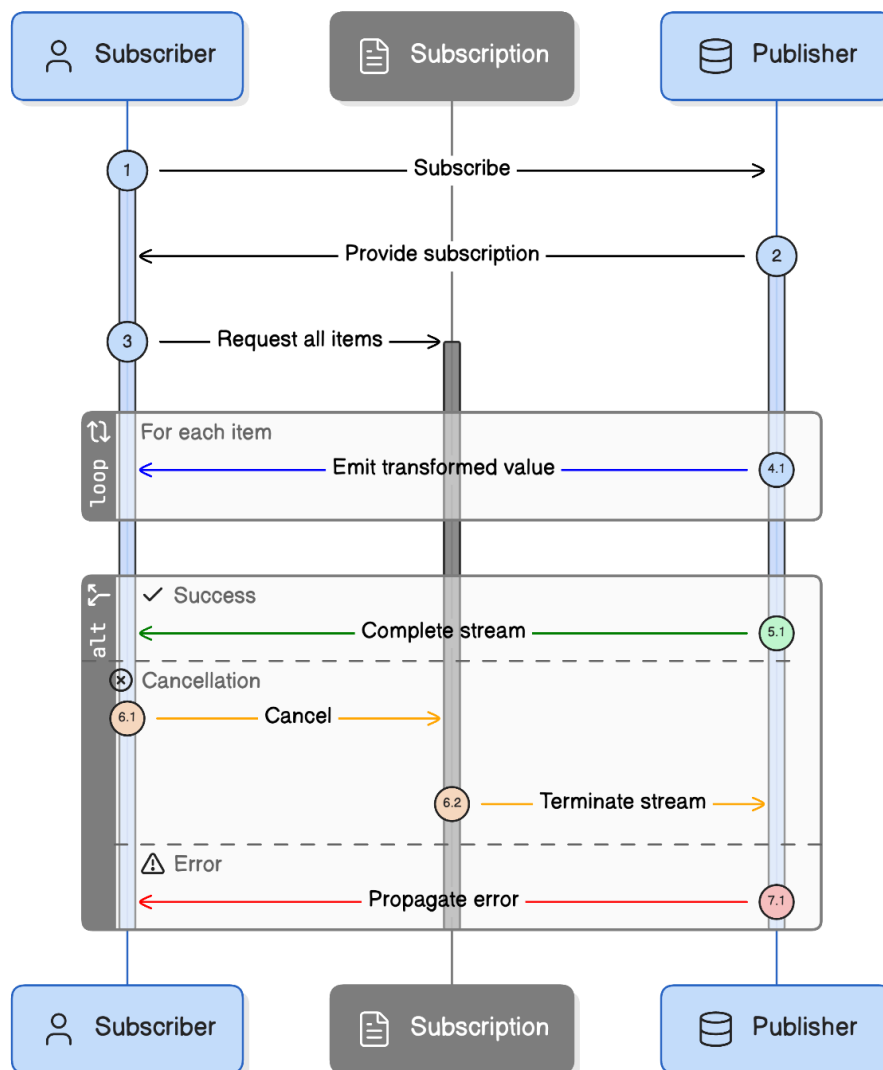
- `onSubscribe` — called exactly once, first, before anything else. The Publisher hands the Subscriber a `Subscription` object. This is the moment the Subscriber gets the remote control.
- `onNext` — called for each item. Can be called zero or more times. Only called after `request(n)` was called.
- `onError` — something went wrong. Stream is terminated. `onNext` will never be called again after this.
- `onComplete` — stream finished successfully. `onNext` will never be called again after this.



`onError` and `onComplete` are mutually exclusive and terminal. One of them fires exactly once. Never both.

```
// This does NOTHING – no DB call happens
Mono<User> query = userRepository.findById(42L);

// THIS triggers the actual DB call
query.subscribe(
    user -> System.out.println("Got: " + user),    // onNext
    error -> System.out.println("Error: " + error), // onError
    () -> System.out.println("Done!")              // onComplete
);
```



Subscription

```

public interface Subscription {
    void request(long n);
    void cancel();
}
  
```

Two methods. This is the backpressure mechanism.

- `request(n)` — the Subscriber says "I am ready for `n` more items." The Publisher must not send more than `n` items until another `request()` call is made.

`Long.MAX_VALUE` means "send everything as fast as you can" — effectively disabling backpressure.

- `cancel()` — the Subscriber says "stop, I don't want any more." The Publisher should stop emitting. This is how `take(5)` works internally, after receiving 5 items, it calls `cancel()` and the source stops producing.

The Subscription object is the only thing the Subscriber holds a reference to. It never has a direct reference to the Publisher.

Processor

```
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> {  
    // no new methods  
    // receives T from upstream Publisher  
    // emits R to downstream Subscriber  
}
```

Every operator in Reactor `map`, `filter`, `flatMap`, `zip` is internally a `Processor`. It subscribes to the upstream, transforms the data, and publishes to the downstream.



You almost never implement `Processor` directly. You use operators which are Processors under the hood.

Stream Lifecycle Hooks

Lifecycle hooks are special operators in **Project Reactor** that let you **peek** into different stages of a stream's lifecycle without modifying the actual data flow. They are very useful for **debugging**, logging, side effects, and monitoring.

1. `doOnNext()`

- **When?** Called every time an item is **emitted** successfully (`onNext`).
- **Use Case:** Logging, processing, or side effects for each item.

2. doOnSubscribe()

- **When?** Called **once**, when the Subscriber subscribes to the stream.
- **Use Case:** Initialize resources, log subscription start, or track when the stream is activated.

3. doFinally()

- **When?** Called **once** at the very end, whether the stream completes **successfully**, with an **error**, or is **cancelled**.
- **Use Case:** Cleanup resources (close files, database connections, etc.).

```
Flux.just("One", "Two", "Three")
    .doOnSubscribe(s -> System.out.println("Subscribed"))
    .doOnNext(item -> System.out.println("→ Processing: " + item))
    .doFinally(signal -> System.out.println("→ Finished with: " + signal))
    .subscribe(
        System.out::println, // onNext
        err -> System.err.println("Error: " + err), // onError
        () -> System.out.println("✅ Done!") // onComplete
    );
```